

Optimizacija upita

Indeksne strukture s B-stablama, implementacije spojeva

Sistemi baza podataka; dr Vladimir Dimitrieski

1

Sadržaj

- Indeksne strukture
- Optimizacija upita
- Naredba *explain plan*

2

2

Indeksne strukture

3

Performanse BP

- Razlozi za loše performanse baze podataka
 - **loše projektovana baza podataka**
 - degradacija performansi usled kontinuiranog razvoja informacionog sistema
 - rast BP,
 - nove putanje pristupa podacima,
 - dodatni korisnici,
 - promene u poslovanju,
 - **neoptimizovani upiti**

4

4

Performanse BP

- Optimizacijom upita često mogu postići veći pozitivni efekti na performanse aplikacije
 - nego optimizacijom hardvera, podešavanjem parametara operativnog sistema ili konfiguracije SQL servera
- Neoptimizovani upit nad bazom podataka može da zauzme veliki broj raspoloživih resursa
 - i na taj način da smanji uticaj uložениh sredstava na performanse čitave aplikacije

5

5

Uzroci loših performansi upita

- Neki od razloga:
 - loše projektovana BP
 - engl. *poor database design*
 - skeniranje tabela
 - nedostatak indeksa i neodgovarajući izbori indeksiranja
 - engl. *poor indexing*
 - ako se ne upotrebljavaju raspoloživi indeksi
 - zastarele statističke informacije BP
 - engl. *inaccurate statistics*
 - neoptimalno spojene tabele i loša specifikacija upita
 - engl. *non-set-based operations, poor query design*
 - previše međusobnog blokiranja transakcija i zastoja
 - engl. *excessive blocking and deadlocks*
 - plan izvršenja upita koji se ne kešira
 - engl. *nonreusable execution plan*

6

6

Datoteke sa podacima i indeksima

- Osnovni cilj sistema baze podataka je da skladišti podatke i omogući efikasan pristup podacima
- Podaci se skladište u **datotekama sa podacima**
 - engl. *data files*
 - koriste se specijalizovani formati datoteka, koji treba da omoguće
 - efikasno skladištenje – minimizacija potrebnog skladišta po jedinici „korisnih“ podataka
 - efikasan pristup – lociranje potrebnih podataka u najmanjem broju koraka
 - efikasnu modifikaciju – modifikacija podataka sa najmanjim brojem izmenjenih memorijskih lokacija na disku
- Pomoćne strukture za ubrzanje pristupa podacima se skladište u **indeksnim datotekama**
 - engl. *index files*
 - pomoćne strukture podataka koje omogućavaju pronalaženje potrebnih podataka na disku bez potrebe za pretraživanjem celokupne datoteke sa podacima
 - bez potrebe za prolaskom kroz celokupnu tabelu sa podacima
- Moguće je da se indeksi i podaci smeštaju u **istoj datoteci**

7

7

Tehnike indeksiranja

- **Indeksi su objekti baze podataka koji obezbeđuju direktan i brz pristup torkama u tabeli**
 - indeks ne menja podatke, on samo predstavlja brzu pristupnu putanju do podatka
 - indeksne datoteke predstavljaju efikasan način smeštanja indeksa na disk
- DBMS automatski održava i koristi indekse
- U praksi, kreiranje indeksa je iterativan proces

8

8

Tehnike indeksiranja

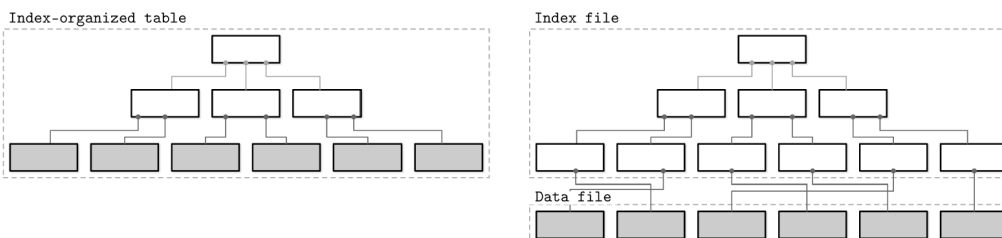
- Kolone nad kojima se kreira indeks zovu se **indeksne** (engl. *key*) **kolone**
 - ukoliko se indeks pravi nad kolonama primarnog ključa, indeks se zove **primarni indeks**
 - engl. *primary index*
 - svi ostali indeksi se zovu **sekundarni indeksi**
 - engl. *secondary index*
- **Klasterizovani indeksi**
 - engl. *clustered index*
 - ukoliko je potrebno da se obezbedi konstantno efikasan pristup podacima prema nekim indeksnim kolonama, moguće je podatke u datotekama sa podacima sortirati prema vrednostima tih kolona
 - klasterizovana datoteka = datoteka sa podacima && indeksna datoteka
 - često, SUBP-ovi omogućavaju postojanje samo jednog klasterizovanog indeksa
 - često, primarni indeksi su klasterizovani dok su sekundarni indeksi po pravilu neklasterizovani

9

9

Tehnike indeksiranja – fizička struktura

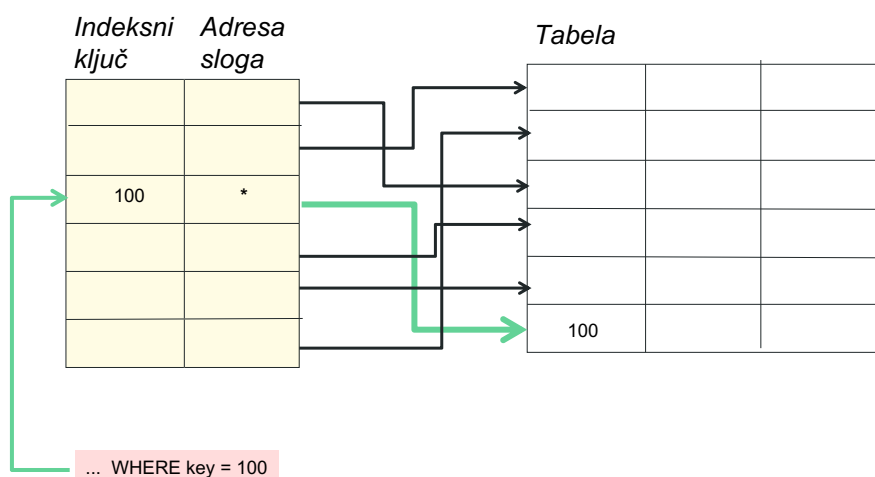
- Dva pristupa fizičkoj organizaciji indeksiranja podataka
 - **indeks je deo datoteke sa podacima**
 - indeksna zona je deo primarne zone sa podacima
 - podaci su listovi u indeksnom stablu
 - **indeks je smešten u odvojenu, indeksnu datoteku**
 - primarna i indeksna zona
 - *napomena:* klasterizovani indeksi mogu slediti oba pristupa



izvor: Alex Petrov: Database internals 10

10

Tehnike indeksiranja – logička struktura

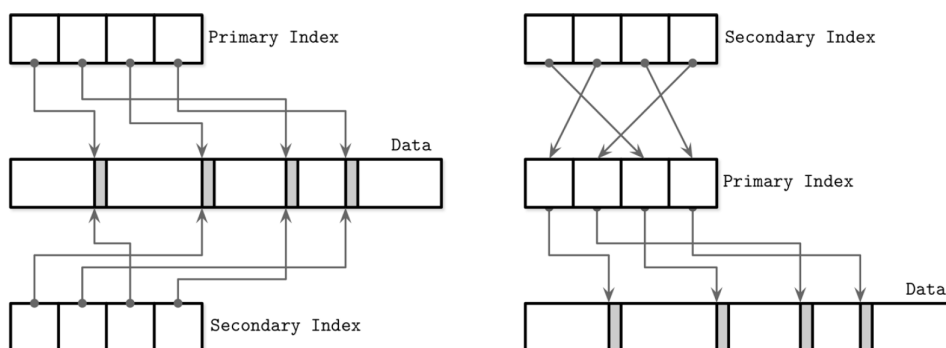


11

11

Tehnike indeksiranja – logička struktura

- Kao adresu originalnog sloga uz vrednosti indeksnih kolona, indeksi mogu sadržati
 - relativnu adresu sloga** u datoteci sa podacima (engl. *RowID*)
 - vrednost primarnog ključa** sloga
 - relativnu adresu sloga i vrednost primarnog ključa**



izvor: Alex Petrov: Database internals 12

12

Tehnike indeksiranja – logička struktura

- Kao adresu originalnog sloga uz vrednosti indeksnih kolona, indeksi mogu sadržati
 - **relativnu adresu sloga** u datoteci sa podacima
 - brži pristup podacima usled manjeg broja čitanja sa diska
 - ažuriranje indeksa prilikom svakog ažuriranja ili premeštanja sloga
 - **vrednost primarnog ključa** sloga
 - manja verovatnoća ažuriranja primarnog ključa u odnosu na relativnu adresu
 - veći broj pristupa disku usled čitanja i blokova primarnog indeksa
 - **relativnu adresu sloga i vrednost primarnog ključa**
 - prvo se proverí relativna adresa sloga
 - ukoliko relativna adresa nije validna, ide se preko primarnog indeksa
 - skuplje ukoliko se često menjaju podaci
 - ažurira se relativna adresa nakon što se pronađe slog posredstvom primarnog indeksa

izvor: Alex Petrov: Database internals 13

13

Tehnike indeksiranja - relativna adresa sloga

```
select rowid, ime from radnik;
```

AAAGKcAAEAAAAGNAAA	Pera
AAAGKcAAEAAAAGNAAB	Milan
AAAGKcAAEAAAAGNAAC	Eva
AAAGKcAAEAAAAGNAAD	Moma
AAAGKcAAEAAAAGNAAE	Rastko



14

14

Tehnike indeksiranja - relativna adresa sloga

```
select rowid, nap from projekat;
```

AAAGKeAAEAAAAGdAAA	Projekat jacanja sistema osiguranja dep.
AAAGKeAAEAAAAGdAAB	Mali Biznis kredit
AAAGKeAAEAAAAGdAAC	Internet bankarstvo
AAAGKeAAEAAAAGdAAD	Osiguranje
AAAGKeAAEAAAAGdAAE	Gotovinski krediti
AAAGKeAAEAAAAGdAAF	Ziro racuni
AAAGKeAAEAAAAGdAAG	Refinansiranje kredita
AAAGKeAAEAAAAGdAAH	Data Warehouse OB

15

15

Tehnike indeksiranja

	Kolona PLT tabele Radnik	Indeksna kolona za PLT u sekundarnom indeksu
<pre>select plt from radnik where plt < 10000;</pre>	10000	6000
	20000	8000
	15000	8700
	8000	9000
	40000	9500
	20000	10000
	35000	10000
	9000	10000
	6000	10000
	32000	15000
	...	20000
	10000	20000
	9500	20000
	8700	20000
	10000	20000
	20000	32000
	10000	32000
	20000	35000
	32000	40000

16

16

Tehnike indeksiranja

- Napomene o indeksiranju
 - Ako se često radi filtriranje slogova na osnovu neke kolone onda se kreira indeks
 - smanjuje broj verifikacija vrednosti
 - Indeks može da održava redosled podataka, iako podaci nisu skladišteni u redosledu tih indeksnih kolona
 - podrazumevana vrednost za indeks je *ascending order*
 - Ako ne postoji potreba za drugim podacima iz tabele onda se i ne pristupa tabeli već samo indeksu
 - Unos, izmena i brisanje torki iz tabele se automatski propagira na relevantne indekse
 - Tabela može imati više indeksa
 - **ne preveliki broj indeksa po tabeli** (maksimalno do 3-4)
 - **zbog propagacije operacija ažuriranja** nad podacima

17

17

Indeksi

- **Indeks se koristi:**
 - kada se kolona često koristi u traženju ili u izrazima upita
 - kada se koristi u spojevima tabela
 - kada je velika selektivnost kolone
 - $\text{selektivnost} = d / n$
 - (broj različitih vrednosti / broj torki tabele)
 - selektivnost se smatra niskom, ako je manja od 20%.
 - za strani ključ
- **Indeks se ne koristi:**
 - kada kolona ili izraz imaju mali broj različitih vrednosti
 - izuzev *bitmap* indeksa u DW
 - kada se često radi *update* date kolone
 - kada se kolone koriste samo u okviru funkcija ili izraza

18

18

Indeksi

- Kreiranje indeksa

```
CREATE [UNIQUE] INDEX index_name
ON table_name (column1, column2, ...) [REVERSE];
```

- Brisanje indeksa

```
DROP INDEX index_name;
```

19

19

Tipovi indeksa

- *Simple index*
 - Kreira se nad jednom kolonom
- *Composite index*
 - Kreira se nad više kolona
- *Unique index*
 - Kreira se nad kolonom/kolonama koja treba da bude unique
- *Reverse index*
 - Ukoliko imamo problem sa nebalansiranošću stabla indeksa usled brisanja torki
- *Function based index*
 - Kreira se nad izrazima ili funkcijama

20

20

Simple index – primer

- Kreirati prosti (*simple*) indeks nad kolonom PLT u tabeli RADNIK:

```
CREATE INDEX idx_plt ON radnik (plt);
```

- Napomene i korisni upiti:

```
select * from tab;
```

- sve tabele i poglede koje imamo u šemi

```
select constraint_name from user_constraints
```

```
where table_name='RADNIK';
```

- sva ograničenja u okviru tabele

```
desc user_indexes;
```

- koje kolone ima tabela koja čuva podatke o indeksima

```
select index_name from user_indexes
```

```
where table_name = 'RADNIK';
```

- provera koji sve indksi postoje

21

21

Composite index

- Kompozitni indeks sadrži više od jedne kolone
- Treba razmotriti kreiranje kompozitnog indeksa **nad kolonama koje se često koriste zajedno** u uslovima WHERE klauzule kombinovanim sa AND operatorom
 - Posebno ako je kombinovana selektivnost bolja od selektivnosti svake kolone pojedinačno
- Ako nekoliko upita selektuje isti skup kolona treba razmotriti o kreiranju kompozitnog indeksa na svim tim kolonama

22

22

Composite index

- Redosled kolona
 - Stavlja se **na prvo mesto kolona sa najviše *unique* vrednosti** jer će to smanjiti skup torki rezultata
- Unapređuje selektivnost
 - Dve ili više kolona sa malom selektivnošću mogu se kombinovati tako da formiraju indeks sa višom selektivnošću
- Redukuje I/O
 - Ako se sve kolone koje se selektuju u okviru upita nalaze u kompozitnom indeksu onda *Oracle* SUBP vraća ove vrednosti bez pristupanja tabeli

23

23

Composite index – primer

- Kreirati kompozitni indeks nad kolonama IME i PRZ u tabeli Radnik:

```
CREATE INDEX ind_cp ON radnik(ime, prz);
```

24

24

Unique index – primeri

- Kreirati jedinstveni (unique) indeks nad kolonom NAP u tabeli Projekat:

```
CREATE UNIQUE INDEX ind_uq ON projekat (nap);
```

- Ovaj indeks obezbeđuje jedinstvenost vrednosti nad indeksiranom kolonom
 - pravi se zbog ograničenja a ne zbog boljih performansi u odnosu na regularan indeks

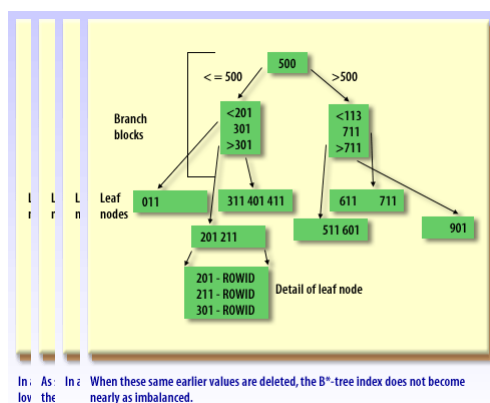
25

25

Reverse index – primeri

- Kreirati reverzni indeks nad kolonom PLT u tabeli Radnik:

```
CREATE INDEX ind_rv ON radnik (mbr) REVERSE;
```



izvor: Relational DB Design: <https://www.relatonaldbdesign.com/>

26

26

Function-based index

- **Izračunava vrednost izraza u kojem učestvuju jedna ili više kolona i smešta ih u indeks**
 - upiti mogu dobiti vrednost izraza iz indeksa umesto da je izračunavaju
- Izraz može biti:
 - aritmetički izraz ili
 - izraz koji sadrži funkcije
 - SQL funkcije
 - PL/SQL funkcije
 - package funkcije
- Indeks ne sme da sadrži null vrednosti
 - koristiti funkciju NVL() ukoliko postoji mogućnost za dobijanjem null vrednosti
- Pogledati:
 - <https://oracle-base.com/articles/8i/function-based-indexes>

27

27

Function based index – primer

- Kreirati indeks koji je pogodan za dati upit:


```
SELECT * FROM radnik
WHERE plt+pre > 50000
```
- Funkcijski indeks nad izrazom (PLT+PRE) u tabeli Radnik:


```
CREATE INDEX ind_fb ON radnik (plt+pre);
```

28

28

Default Indeksi

- Oracle SUBP implicitno kreira i koristi indekse kada se definišu sledeća ograničenja:
 - ograničenje primarnog ključa i
 - ograničenje jedinstvenosti vrednosti obeležja

```
SELECT index_name
FROM user_indexes
WHERE table_name = 'PROJEKAT';
```

29

29

Indeksi i strani ključ

- Indeks se ne kreira automatski na stranom ključu
- **Ako se kolone stranog ključa često koriste u uslovima spajanja onda treba kreirati ključ na tim kolonama**
- DML operacije se takođe unapređuju ovom vrstom indeksa
 - Brisanje ili izmena torke iz tabele roditelja zahteva proveru da ne postoji ni jedna torka u tabeli potomka koja je referencira
 - Bez indeksa mora se proći kroz čitavu tabelu potomka

30

30

Indeksi i strani ključ

- Kreirati indeks na koloni RUK u tabeli projekat.

```
CREATE INDEX ind_fk ON projekat(ruk);
```

31

31

Optimizacija upita

32

Optimizator

- Faktori koje razmatra optimizator:
 - Kardinalitet (engl. *cardinality*)
 - Pristupni put (engl. *access path*)
 - Metod spajanja (engl. *join method*)
 - Redosled spajanja (engl. *join order*)
 - Paralelizacija izvršavanja (engl. *parallel execution*)

33

33

Optimizator

- Biranje pristupnog puta
 - Dostupni pristupni putevi za naredbu
 - Procena koštanja izvršavanja naredbe korišćenjem svakog od pristupnih puteva
 - generiše skup mogućih planova izvršavanja
 - procenjuje koštanje svakog plana
 - koristi staru statistiku (LAST_ANALYZED, BLOCKS u ALL_TABLES)
 - bira plan sa najnižom cenom
 - *cost-based optimizer*
 - I/O, CPU, korišćenje mrežnih resursa
 - Konstruiše **matricu redosleda spajanja i metoda spajanja** i pridružuje cenu svakom paru u matrici
 - tabela prikazuje moguće varijante metoda i redosleda spajanja sa cenom
 - na osnovu toga odlučuje i bira opciju sa najmanjom cenom

34

34

Optimizator

- Mogući pristupni putevi (engl. *Access path*)
 - *Full table scans*
 - *Row ID scans*
 - *Index scans*

35

35

Access path - Full table scans

- Čita sve torke iz tabele i onda filtrira one koji zadovoljavaju uslov selekcije
- *Full table scan* se koristi kada imamo:
 - nedostatak indeksa
 - veliku količinu podataka
 - malu tabelu
- Ako optimizator misli da mu treba većina blokova onda će birati *full table scan* bez obzira na indeks

36

36

Access path - Full table scans

- Skenira sve blokove iz tabele koji su ispod *high-water mark* (HWM)
 - HWM označava količinu korišćenog prostora ili prostora koji je formatiran da prihvati podatke
- Blokovi se čitaju sekvencijalno
- Pošto su blokovi susedni može se nekada koristiti povećanje bloka u I/O pozivima
 - Inicijalizacijom parametra `DB_FILE_MULTIBLOCK_READ_COUNT`
 - Specificira maksimalni broj blokova u jednoj I/O operaciji
 - Zavisí od platforme na kojoj je instaliran SUBP

37

37

Full table scans

```
SELECT last_analyzed, blocks
FROM all_tables
WHERE table_name='RADNIK';
```

- Ako tabela nije analizirana od kada je kreirana i mala je optimizator će birati *full table scans*
- Ručno pokretanje sakupljanja statistike

```
ANALYZE TABLE Radnik COMPUTE STATISTICS;
```

38

38

Full table scans – primer

- *Full table scans*

```
SELECT plt FROM radnik
WHERE plt > 25000;
```

Full table scans – primer

	Id		Operation		Name		Rows		Bytes		Cost (%CPU)		Time	
	0		SELECT STATEMENT				8		104		3 (0)		00:00:01	
	* 1		TABLE ACCESS FULL		RADNIK		8		104		3 (0)		00:00:01	

Predicate Information (identified by operation id):

```
1 - filter("PLT">25000)
```

Access path

- *Row ID scans*
 - Row ID specificira datoteku i blok u kojem se nalazi tražena torke kao i lokaciju torke u tom bloku
 - Korišćenje Row ID je najbrži način za dobijanje jedne torke
 - Skeniranje indeksa ne podrazumeva uvek pristup tabeli pomoću Row ID
 - npr. ukoliko su sve tražene kolone deo indeksa

41

41

Row ID scans – primer

```
SELECT * FROM radnik  
WHERE mbr > 190;
```

42

42

Row ID scans – primer

Plan hash value: 243654634

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		4	124	2 (0)	00:00:01
1	TABLE ACCESS BY INDEX ROWID	RADNIK	4	124	2 (0)	00
* 2	INDEX RANGE SCAN	RADNIK_PK	4		1 (0)	00:00:01

Predicate Information (identified by operation id):

2 - access("MBR">190)

43

43

Access path

- *Index scans*
 - *Index Range scan*
 - *Index Full scan*
 - *Index Fast-full scan*
 - *Index Join scan*
 - *Index Skip scan*

44

44

Index Range Scan

- *Index Range Scan*
 - Skeniranje opsega vrednosti u redosledu sortiranih vrednosti indeksa

```
SELECT plt FROM radnik
WHERE plt > 25000;
```

- Napomena:
 - prethodno je kreiran indeks
 - `CREATE INDEX idx_plt ON radnik (plt);`

45

45

Index Range Scan

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		8	104	1 (0)	00:00:01
* 1	INDEX RANGE SCAN	IDX_PLT	8	104	1 (0)	00:00:01

Predicate Information (identified by operation id):

```
1 - access("PLT">25000)
```

46

46

Index Full scan

- *Index Full scan* čita ceo indeks redom u kojem su podaci sortirani
 - zapravo čita **samo listove stabla i prvu granu stabla** potrebnu radi pronalaženja prvog lista
 - koristi operacije čitanja pojedinačnog bloka sa diska
- Može da eliminiše operacije sortiranja (ORDER BY) zato što je indeks sortiran po indeksnoj koloni
 - ukoliko su sve kolone iz ove klauzule u indeksu i poredane su na isti način
- Koristi se u sledećim situacijama:
 - Predikat referencira kolonu u indeksu
 - Ne mora biti vodeća kolona.
 - Nije specificiran predikat ali postoje sledeći uslovi:
 - Sve kolone u tabeli koje su u upitu su i u indeksu
 - Najmanje jedna indeksirana kolona nije *null*

47

47

Index Full scan – primer

- *Index Full scan*

```
SELECT spr, nar
FROM projekat
ORDER BY spr;
```

48

48

Index Full scan – primer

Plan hash value: 2118842401

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		9	198	2 (0)	00:00:01
1	TABLE ACCESS BY INDEX ROWID	PROJEKAT	9	198		
2	INDEX FULL SCAN	PROJEKAT_PK	9		1 (0)	00:00:01

49

49

Index fast full scan

- *Index fast full scan*
 - Čita blokove indeksa u nesortiranom redosledu, kako su smešteni na disku.
 - koristi operacije istovremenog čitanja više blokova
 - omogućava brže čitanje
 - Korsiti se kada se čita indeks umesto tabele, koristi indeks kao da je tabela
 - Optimizator razmatra ovo skeniranje kada se u upitu pristupa samo atributima koji su u indeksu
 - Ne može da eliminiše operacije sortiranja zato što ne čita indeks u sortiranom redosledu

50

50

Index Join Scans

- *Index join scan* spaja više indeksa zajedno i vraća sve kolone zahtevane upitom
 - nije potrebno pristupanje tabelama zato što se svi podaci dobijaju iz indeksa
 - u slučaju da odvojeni indeksi imaju sve kolone koje se nalaze u SELECT klauzuli
 - vrši se *hash join* vrednosti u indeksima

51

51

Index Skip scan

- *Index skip scan* se dešava kada inicijalna kolona kompozitnog indeksa nije specificirana u upitu.
- Primer:

```
SELECT plt
FROM radnik WHERE PRZ='Peric';
```
- **Napomena:** Mora postojati kreiran indeks nad ime, prz

```
CREATE INDEX idx_imeprz ON radnik (ime, prz);
```

52

52

Index Skip scan – primer

Plan hash value: 2538798176

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		1	11	2 (0)	00:00:01
1	TABLE ACCESS BY INDEX ROWID	RADNIK	1	11	2 (0)	00:00:01
* 2	INDEX SKIP SCAN	IND_CP	1		1 (0)	00:00:01

Predicate Information (identified by operation id):

```

2 - access("PRZ"='Peric')
    filter("PRZ"='Peric')

```

53

53

Spajanje više tabela – Join

- Mogu se spojiti samo dva izvora torki u isto vreme. Spajanje više od dve tabele vrši se na sledeći način:
 - spajanjem dve tabele dobija se izvor torki
 - sledeća tabela se spaja sa izvorom torki koji je rezultat iz koraka jedan
 - korak 2 se ponavlja dok se ne spoje sve torke

54

54

Join terminologija

- Join naredba
- Join predikat i nonjoin predikat
- Single-row predikat

```
SELECT p.spr, p.nap, r.ime, r.prz
FROM projekat p, radnik r
WHERE p.ruk=r.mbr      ← Join predikat
AND (r.mbr=100        ← Nonjoin predikat
OR p.spr=10);         ← Single-row predikat
```

55

55

Optimizator

- Redosled spajanja
 - Pravilo 1. *Single-row* predikat forsira da njegov izvor torki bude prvi u redosledu spajanja
 - Pravilo 2. Tabela sa outer join mora da bude poslednja u redosledu spajanja
- Primeri (pravilo 1):

```
SELECT * FROM projekat, radproj
WHERE projekat.spr = radproj.spr
```

```
SELECT * FROM projekat, radproj
WHERE projekat.spr = radproj.spr AND projekat.nap = 'IIS'
```

56

56

Optimizator

- Vrste implementacije spojeva
 - *Nested loop join*
 - *Hash join*
 - *Sort-merge join*
 - *Cartesian join*

57

57

Join method

- *Nested loop join*
 - Jedna od dve tabele se proglašava spoljnom (engl. *outer table*) ili vodećom tabelom (engl. *driving table*)
 - Druga tabela je nazvana unutrašnjom tabelom (engl. *inner table*)
 - Za svaku torku spoljašnje tabele vraćaju se sve odgovarajuće torke unutrašnje tabele
 - Pogodne kada se spajaju mali podskupovi podataka i kada postoji efikasan način za pristupanje drugoj tabeli
 - npr. indeks nad kolonom unutrašnje tabele, nad kojom se vrši spajanje
- hintovi optimizatoru
 - `/*+ ..... */`
 - `USE_NL(table)`
 - `USE_NL_WITH_INDEX(table index)`

58

58

Nested loop join

- Konceptualno *nested loops join* ekvivalentan je sledećem pseudokodu
 - sa dve ugnježdene iteracije

```
FOR erow IN (SELECT * FROM employees WHERE X=Y)
LOOP
  FOR drow IN (SELECT * FROM departments WHERE erow is matched)
  LOOP
    OUTPUT values from erow and drow
  END LOOP
END LOOP
```

59

59

Join method

- *Hash join*
 - Koristi se kada je u pitanju spajanje po jednakosti velikih skupova podataka
 - Optimizator koristi manji skup podataka od dva za pravljenje hash tabele na koloni za spajanje
 - u memoriji, koristeći determinističku hash funkciju za specificiranje lokacije u hash tabeli u kojoj je smeštena svaka torka
 - Zatim se skenira veći skup podataka da nađe torke za spajanje
- hintovi optimizatoru
 - `USE_HASH(table)`

60

60

Hash join

```

FOR small_table_row IN (SELECT * FROM small_table) LOOP
    slot_number := HASH(small_table_row.join_key);
    INSERT_HASH_TABLE(slot_number, small_table_row);
END LOOP;
-----
FOR large_table_row IN (SELECT * FROM large_table) LOOP
    slot_number := HASH(large_table_row.join_key);
    small_table_row =
        LOOKUP_HASH_TABLE(slot_number, large_table_row.join_key);
    IF small_table_row FOUND THEN
        OUTPUT small_table_row + large_table_row;
    END IF;
END LOOP;

```

61

61

Join method

- *Sort-merge join*
 - Torke iz oba izvora torki su sortirane na kolonama join predikata
 - Dva sortirana izvora torki se spajaju (*merge*) u rezultujući izvor torki
- hintovi optimizatoru
 - NO_INDEX and USE_MERGE

62

62

Sort-merge join

- Optimizator može izabrati sort merge join umesto hash join za spajanje velike količine podataka kada postoji neki od uslova:
 - uslov spajanja između dve tabele nije spajanje po jednakosti, već se koristi uslovi kao što su $<$, $<=$, $>$, or $>=$
 - za razliku od *sort merge*, ***hash join* zahteva uslov spajanja po jednakosti**
 - zbog zahteva za sortiranjem drugih operacija, optimizator nalazi da je jeftinije da koristi *sort merge*
 - ako postoji indeks tada se ne mora sortirati prvi skup podataka jer je već sortiran, dok se uvek sortira drugi skup podataka bez obzira na indeks

63

63

Naredba *explain plan*

64

Plan izvršavanja

- Plan izvršavanja je skup koraka koje obavlja optimizator u izvršavanju SQL naredbi
 - uključuje metodu pristupa svakoj tabeli kojoj se pristupa iz naredbe i redosled tabela (*join order*)
 - koristi različite metode za kombinovanje torki iz različitih tabela (*join method*)
 - koraci u planu izvršavanja se ne obavljaju u redosledu u kojem su numerisani
- Metod za gledanje plana izvršavanja
 - EXPLAIN PLAN

65

65

Plan izvršavanja

- Naredba EXPLAIN PLAN
 - Generiše plan izvršavanja optimizatora
 - Čuva plan u tabeli PLAN_TABLE
 - Ne izvršava naredbu

- Sintaksa

```
EXPLAIN PLAN
[SET STATEMENT_ID = 'text']
[INTO korisnička tabela za plan]
FOR SQL naredba
```

66

66

Plan izvršavanja

```
[SET STATEMENT_ID = 'text']
```

- Zadaje se identifikator naredbe radi kasnijeg korišćenja
- Posebno je pogodno:
 - kada se deli tabela planova sa drugim korisnicima
 - kada se čuva više planova izvršavanja u istoj tabeli planova

```
[INTO korisnička tabela za plan]
```

- Korisnik može zadati sopstvenu tabelu planova
 - podrazumevana je PLAN_TABLE

67

67

Primer EXPLAIN PLAN

```
EXPLAIN PLAN
SET STATEMENT_ID = 'primer1'
FOR
SELECT p.spr, p.nap, r.ime, r.prz
FROM projekat p, radnik r
WHERE p.ruk = r.mbr;
```

- Rezultat izvršenja naredbe:
Explained. ili Plan SET succeeded.

68

68

Primer EXPLAIN PLAN - output

```
SELECT PLAN_TABLE_OUTPUT
FROM TABLE (DBMS_XPLAN.DISPLAY());
```

```
SELECT PLAN_TABLE_OUTPUT
FROM TABLE (DBMS_XPLAN.DISPLAY('plan_table','primer1'));
```

- DBMS_XPLAN paket
 - funkcija DISPLAY koristi se za formatiranje i prikazivanje poslednje naredbe smeštene u tabeli planova

69

69

Primer EXPLAIN PLAN - output

```
SELECT PLAN_TABLE_OUTPUT FROM TABLE (DBMS_XPLAN.DISPLAY('plan_table','primer1'));
```

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		9	378	6 (17)	00:00:01
1	MERGE JOIN		9	378	6 (17)	00:00:01
2	TABLE ACCESS BY INDEX ROWID	RADNIK	23	368	2 (0)	00:00:01
3	INDEX FULL SCAN	RADNIK_PK	23		1 (0)	00:00:01
4	SORT JOIN		9	234	4 (25)	00:00:01
5	TABLE ACCESS FULL	PROJEKAT	9	234	3 (0)	00:00:01

Predicate Information (identified by operation id):

```

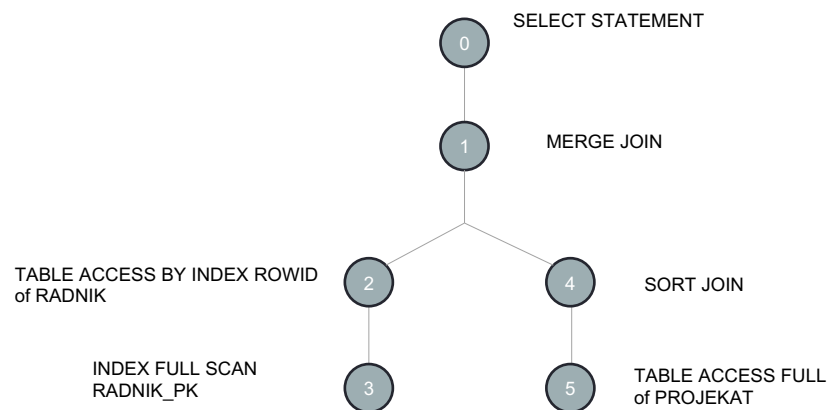
4 - access ("P"."RUK"="R"."MBR")
    filter ("P"."RUK"="R"."MBR")

```

70

70

Primer – stablo izvršavanja (Parse tree)



71

71

Primer 2 EXPLAIN PLAN

```

EXPLAIN PLAN
SET STATEMENT_ID = 'primer2'
FOR
SELECT p.spr, p.nap, r.ime, r.prz
FROM projekat p, radnik r
WHERE p.ruk = r.mbr AND p.nar = 'KBC';

SELECT PLAN_TABLE_OUTPUT FROM TABLE (DBMS_XPLAN.DISPLAY('plan_table','primer2'));
  
```

72

72

Primer 2 EXPLAIN PLAN

```
SELECT PLAN_TABLE_OUTPUT FROM TABLE (DBMS_XPLAN.DISPLAY('plan_table','primer2'));
```

PLAN_TABLE_OUTPUT									
1	Plan hash value: 1892854466								
2									
3	-----								
4	Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time		
5	-----								
6	0	SELECT STATEMENT		1	49	4 (0)	00:00:01		
7	1	NESTED LOOPS							
8	2	NESTED LOOPS		1	49	4 (0)	00:00:01		
9	* 3	TABLE ACCESS FULL	PROJEKAT	1	33	3 (0)	00:00:01		
10	* 4	INDEX UNIQUE SCAN	RADNIK_PK	1		0 (0)	00:00:01		
11	5	TABLE ACCESS BY INDEX ROWID	RADNIK	1	16	1 (0)	00:00:01		
12	-----								
13									
14	Predicate Information (identified by operation id):								
15	-----								
16									
17	3	- filter("P"."NAR"='OB')							
18	4	- access("P"."RUK"="R"."MBR")							

73

73

Primer 3 EXPLAIN PLAN

```
EXPLAIN PLAN
SET STATEMENT_ID = 'primer3'
FOR
SELECT r.ime, r.prz, rp.spr
FROM radnik r, radproj rp
WHERE r.mbr=rp.mbr;

SELECT PLAN_TABLE_OUTPUT FROM TABLE (DBMS_XPLAN.DISPLAY('plan_table','primer3'));
```

74

74

Primer 3 EXPLAIN PLAN

```
SELECT PLAN_TABLE_OUTPUT FROM TABLE (DBMS_XPLAN.DISPLAY('plan_table','primer3'));
```

Script Output x Query Result x

SQL | All Rows Fetched: 18 in 0.031 seconds

PLAN_TABLE_OUTPUT							
1	Plan hash value:	2113081822					
2							
3	-----						
4	Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
5	-----						
6	0	SELECT STATEMENT		25	575	4 (25)	00:00:01
7	1	MERGE JOIN		25	575	4 (25)	00:00:01
8	2	TABLE ACCESS BY INDEX ROWID	RADNIK	23	368	2 (0)	00:00:01
9	3	INDEX FULL SCAN	RADNIK_PK	23		1 (0)	00:00:01
10	* 4	SORT JOIN		25	175	2 (50)	00:00:01
11	5	INDEX FULL SCAN	RADPROJ_PK	25	175	1 (0)	00:00:01
12	-----						
13							
14	Predicate Information (identified by operation id):						
15	-----						
16							
17	4	- access("R"."MBR"="RP"."MBR")					
18		filter("R"."MBR"="RP"."MBR")					

75

75

Primer 4 EXPLAIN PLAN

```
EXPLAIN PLAN
SET STATEMENT_ID = 'primer4'
FOR
SELECT r.ime, r.prz, rp.spr, rp.brc
FROM radnik r, radproj rp
WHERE r.mbr=rp.mbr;

SELECT PLAN_TABLE_OUTPUT FROM TABLE (DBMS_XPLAN.DISPLAY('plan_table','primer4'));
```

76

76

Primer 4 EXPLAIN PLAN

```
SELECT PLAN_TABLE_OUTPUT FROM TABLE (DBMS_XPLAN.DISPLAY('plan_table','primer4'));
```

Script Output x Query Result x

SQL All Rows Fetched: 18 in 0.032 seconds

PLAN_TABLE_OUTPUT							
1	Plan hash value: 1180059057						
2							
3	-----						
4	Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
5	-----						
6	0	SELECT STATEMENT		25	625	6 (17)	00:00:01
7	1	MERGE JOIN		25	625	6 (17)	00:00:01
8	2	TABLE ACCESS BY INDEX ROWID	RADNIK	23	368	2 (0)	00:00:01
9	3	INDEX FULL SCAN	RADNIK_PK	23		1 (0)	00:00:01
10	* 4	SORT JOIN		25	225	4 (25)	00:00:01
11	5	TABLE ACCESS FULL	RADPROJ	25	225	3 (0)	00:00:01
12	-----						
13							
14	Predicate Information (identified by operation id):						
15	-----						
16							
17	4	- access("R"."MBR"="RP"."MBR")					
18		filter("R"."MBR"="RP"."MBR")					

77

77

Primer 5 EXPLAIN PLAN

EXPLAIN PLAN

SET STATEMENT_ID = 'primer5'

FOR

SELECT r.ime, r.prz, p.spr, p.nap

FROM radnik r, radproj rp, projekat p

WHERE r.mbr=rp.mbr AND rp.spr=p.spr;

SELECT PLAN_TABLE_OUTPUT FROM TABLE (DBMS_XPLAN.DISPLAY('plan_table','primer5'));

78

78

Primer 5 EXPLAIN PLAN

```
SELECT PLAN_TABLE_OUTPUT FROM TABLE (DBMS_XPLAN.DISPLAY('plan_table','primer5'));
```

Script Output x Query Result x

SQL All Rows Fetched: 21 in 0.047 seconds

PLAN_TABLE_OUTPUT

1 Plan hash value: 2887891508

2

3 -----

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		25	1125	8 (25)	00:00:01
1	HASH JOIN		25	1125	8 (25)	00:00:01
2	MERGE JOIN		25	725	4 (25)	00:00:01
3	TABLE ACCESS BY INDEX ROWID	PROJEKAT	9	198	2 (0)	00:00:01
4	INDEX FULL SCAN	PROJEKAT_PK	9		1 (0)	00:00:01
5	SORT JOIN		25	175	2 (50)	00:00:01
6	INDEX FULL SCAN	RADPROJ_FK	25	175	1 (0)	00:00:01
7	TABLE ACCESS FULL	RADNIK	23	368	3 (0)	00:00:01

14 -----

15

16 Predicate Information (identified by operation id):

17 -----

18

19 1 - access("R"."MBR"="RP"."MBR")

20 5 - access("RP"."SPR"="P"."SPR")

21 filter("RP"."SPR"="P"."SPR")

79

79

Primer 6 EXPLAIN PLAN

```
EXPLAIN PLAN
SET STATEMENT_ID = 'primer6'
FOR
SELECT r.ime, r.prz, p.spr, p.nap
FROM radnik r, radproj rp, projekat p
WHERE r.mbr=rp.mbr AND rp.spr=p.spr and r.mbr=100;

SELECT PLAN_TABLE_OUTPUT FROM TABLE (DBMS_XPLAN.DISPLAY('plan_table','primer6'));
```

80

80

Primer 6 EXPLAIN PLAN

Script Output x Query Result x	
SQL All Rows Fetched: 23 in 0.031 seconds	
PLAN_TABLE_OUTPUT	
2	
3	
4	-----
5	Id Operation Name Rows Bytes Cost (%CPU) Time
6	-----
7	0 SELECT STATEMENT 2 90 4 (0) 00:00:01
8	1 NESTED LOOPS
9	2 NESTED LOOPS 2 90 4 (0) 00:00:01
10	3 NESTED LOOPS 2 46 2 (0) 00:00:01
11	4 TABLE ACCESS BY INDEX ROWID RADNIK 1 16 1 (0) 00:00:01
12	* 5 INDEX UNIQUE SCAN RADNIK_PK 1 0 (0) 00:00:01
13	* 6 INDEX SKIP SCAN RADPROJ_PK 2 14 1 (0) 00:00:01
14	* 7 INDEX UNIQUE SCAN PROJEKAT_PK 1 0 (0) 00:00:01
15	8 TABLE ACCESS BY INDEX ROWID PROJEKAT 1 22 1 (0) 00:00:01
16	-----
17	Predicate Information (identified by operation id):
18	-----
19	
20	5 - access("R"."MBR"=100)
21	6 - access("RP"."MBR"=100)
22	filter("RP"."MBR"=100)
23	7 - access("RP"."SPR"="P"."SPR")

81

81

Primer 7 EXPLAIN PLAN

```

EXPLAIN PLAN
SET STATEMENT_ID = 'primer7'
FOR
SELECT r.ime, r.prz, p.spr, p.nap
FROM radnik r, radproj rp, projekat p
WHERE r.mbr=rp.mbr AND rp.spr=p.spr and r.plt<30000;

SELECT PLAN_TABLE_OUTPUT FROM TABLE (DBMS_XPLAN.DISPLAY('plan_table','primer7'));

```

82

82

Primer 7 EXPLAIN PLAN

```

SELECT PLAN_TABLE_OUTPUT FROM TABLE (DBMS_XPLAN.DISPLAY('plan_table','primer7'));

```

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		24	1728	9 (23)	00:00:01
1	HASH JOIN		24	1728	9 (23)	00:00:01
2	MERGE JOIN		24	1008	5 (20)	00:00:01
3	TABLE ACCESS BY INDEX ROWID	RADNIK	16	256	2 (0)	00:00:01
4	INDEX FULL SCAN	RADNIK_PK	22		1 (0)	00:00:01
5	SORT JOIN		25	650	3 (34)	00:00:01
6	INDEX FAST FULL SCAN	RADPROJ_PK	25	650	2 (0)	00:00:01
7	TABLE ACCESS FULL	PROJEKAT	8	240	3 (0)	00:00:01

Predicate Information (identified by operation id):

```

1 - access("R"."SPR"="P"."SPR")
3 - filter("R"."PLT"<30000)
5 - access("R"."MBR"="R"."MBR")
   filter("R"."MBR"="R"."MBR")

```

Note

```

- dynamic sampling used for this statement (level=2)

```

83

83

Primer 8 - razlike između ugneždenog upita i spoja

```

EXPLAIN PLAN
SET STATEMENT_ID = 'primer8_spoj'
FOR
SELECT DISTINCT r.ime, r.prz
FROM radnik r, projekat p
WHERE r.mbr = p.ruk;

SELECT PLAN_TABLE_OUTPUT
FROM TABLE (DBMS_XPLAN.DISPLAY('plan_table','primer8_spoj'));

```

84

84

Primer 8 - razlike između ugneždenog upita i spoja

```
SELECT PLAN_TABLE_OUTPUT FROM TABLE (DBMS_XPLAN.DISPLAY('plan_table','primer8_spoj'));
```

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		8	208	7 (29)	00:00:01
1	HASH UNIQUE		8	208	7 (29)	00:00:01
2	MERGE JOIN		8	208	6 (17)	00:00:01
3	TABLE ACCESS BY INDEX ROWID	RADNIK	22	286	2 (0)	00:00:01
4	INDEX FULL SCAN	RADNIK_PK	22		1 (0)	00:00:01
5	SORT JOIN		8	104	4 (25)	00:00:01
6	TABLE ACCESS FULL	PROJEKAT	8	104	3 (0)	00:00:01

Predicate Information (identified by operation id):

5 - access("R"."MBR"="P"."RUK")
 filter("R"."MBR"="P"."RUK")

Note

- dynamic sampling used for this statement (level=2)

85

85

Primer 8 - razlike između ugneždenog upita i spoja

```
EXPLAIN PLAN
SET STATEMENT_ID = 'primer8_ugn'
FOR
SELECT r.ime, r.prz
FROM radnik r
WHERE r.mbr in (SELECT ruk FROM projekat);

SELECT PLAN_TABLE_OUTPUT FROM TABLE (DBMS_XPLAN.DISPLAY('plan_table','primer8_ugn'));
```

86

86

Primer 8 - razlike između ugneždenog upita i spoja

```
SELECT PLAN_TABLE_OUTPUT FROM TABLE (DBMS_XPLAN.DISPLAY('plan_table','primer8_uqn'));
```

PLAN_TABLE_OUTPUT

1 Plan hash value: 4292270174

2

3

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		3	78	6 (17)	00:00:01
1	MERGE JOIN SEMI		3	78	6 (17)	00:00:01
2	TABLE ACCESS BY INDEX ROWID	RADNIK	22	286	2 (0)	00:00:01
3	INDEX FULL SCAN	RADNIK_PK	22		1 (0)	00:00:01
4	SORT UNIQUE		8	104	4 (25)	00:00:01
5	TABLE ACCESS FULL	PROJEKAT	8	104	3 (0)	00:00:01

14 Predicate Information (identified by operation id):

15

16

17 4 - access("R"."MBR"="RUK")

18 filter("R"."MBR"="RUK")

19

20 Note

21 -----

22 - dynamic sampling used for this statement (level=2)

87

87

Primer 8 - Indeksi i strani ključ

- Kreirati indeks na ruk u tabeli projekat.

```
CREATE INDEX ind_fk ON projekat(ruk);
```

88

88

Indeksi i strani ključ

```

EXPLAIN PLAN
SET STATEMENT_ID = 'primer9_spoj'
FOR
SELECT DISTINCT r.ime, r.prz
FROM radnik r, projekat p
WHERE r.mbr = p.ruk;

SELECT PLAN_TABLE_OUTPUT FROM TABLE(DBMS_XPLAN.DISPLAY('plan_table','primer9_spoj'));

```

89

89

Indeksi i strani ključ

SELECT PLAN_TABLE_OUTPUT FROM TABLE (DBMS_XPLAN.DISPLAY('plan_table','primer9_spoj'));																			
Script Output x Autotrace x Explain Plan x Query Result x Query Result 1 x Query Result 2 x																			
All Rows Fetched: 20 in 0.02 seconds																			
PLAN_TABLE_OUTPUT																			
1	Plan hash value: 301486184																		
2																			
3																			
4	Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time												
5																			
6	0	SELECT STATEMENT		8	208	4 (25)	00:00:01												
7	1	HASH UNIQUE		8	208	4 (25)	00:00:01												
8	2	NESTED LOOPS		8	208	3 (0)	00:00:01												
9	3	TABLE ACCESS FULL	RADNIK	22	286	3 (0)	00:00:01												
10	* 4	INDEX RANGE SCAN	IND_FK	1	13	0 (0)	00:00:01												
11																			
12																			
13	Predicate Information (identified by operation id):																		
14																			
15																			
16	4	access("R"."MBR"="P"."RUK")																	
17																			
18	Note																		
19	-----																		
20	- dynamic sampling used for this statement (level=2)																		

90

90

Indeksi i strani ključ (spoj sa indeksom i bez indeksa)

<pre>SELECT PLAN_TABLE_OUTPUT FROM TABLE (DBMS_XPLAN.DISPLAY('plan_table','primer9_sj</pre>		<pre>SELECT PLAN_TABLE_OUTPUT FROM TABLE (DBMS_XPLAN.DISPLAY('plan_table','primer8_spoj'));</pre>	
Script Output x Autotrace x Explain Plan x Query Result x Query Result 1 x Query Result 2 x		Script Output x Autotrace x Explain Plan x Query Result x Query Result 1 x Query Result 2 x	
All Rows Fetched: 20 in 0.02 seconds		All Rows Fetched: 23 in 0.02 seconds	
<pre>PLAN_TABLE_OUTPUT 1 Plan hash value: 301486184 2 3 4 Id Operation Name Rows Bytes Cost (%CPU) Time 5 ----- 6 0 SELECT STATEMENT 7 1 HASH UNIQUE 8 208 4 (25) 00:00:01 8 2 NESTED LOOPS 8 208 3 (0) 00:00:01 9 3 TABLE ACCESS FULL RADNIK 22 286 3 (0) 00:00:01 10 * 4 INDEX RANGE SCAN IND_FK 1 13 0 (0) 00:00:01 11 ----- 12 13 Predicate Information (identified by operation id): 14 ----- 15 16 4 - access("R"."MBR"="P"."RUK") 17 18 Note 19 ---- 20 - dynamic sampling used for this statement (level=2)</pre>		<pre>PLAN_TABLE_OUTPUT 1 Plan hash value: 2601328627 2 3 4 Id Operation Name Rows Bytes Cost (%CPU) Time 5 ----- 6 0 SELECT STATEMENT 7 1 HASH UNIQUE 8 208 7 (29) 00:00:01 8 2 MERGE JOIN 8 208 6 (17) 00:00:01 9 3 TABLE ACCESS BY INDEX ROWID RADNIK 22 286 2 (0) 00:00:01 10 4 INDEX FULL SCAN RADNIK_FK 22 1 (0) 00:00:01 11 * 5 SORT JOIN 8 104 4 (25) 00:00:01 12 6 TABLE ACCESS FULL PROJEKAT 8 104 3 (0) 00:00:01 13 ----- 14 15 Predicate Information (identified by operation id): 16 ----- 17 18 5 - access("R"."MBR"="P"."RUK") 19 filter("R"."MBR"="P"."RUK") 20 21 Note 22 ---- 23 - dynamic sampling used for this statement (level=2)</pre>	

91

Indeksi i strani ključ

```
EXPLAIN PLAN
SET STATEMENT_ID = 'primer9_ugn'
FOR
SELECT r.ime, r.prz
FROM radnik r
WHERE r.mbr in (SELECT ruk FROM projekat);

SELECT PLAN_TABLE_OUTPUT FROM TABLE (DBMS_XPLAN.DISPLAY('plan_table','primer9_ugn'));
```

92

92

Indeksi i strani ključ

```
SELECT PLAN_TABLE_OUTPUT FROM TABLE (DBMS_XPLAN.DISPLAY('plan_table','primer9_ugn'));
```

PLAN_TABLE_OUTPUT									
1 Plan hash value: 166315348									
2									
3									
4									
5									
6	0	SELECT STATEMENT		3	78	3	(0)	00:00:01	
7	1	NESTED LOOPS SEMI		3	78	3	(0)	00:00:01	
8	2	TABLE ACCESS FULL	RADNIK	22	286	3	(0)	00:00:01	
9	3	INDEX RANGE SCAN	IND_FK	1	13	0	(0)	00:00:01	
10									
11									
12 Predicate Information (identified by operation id):									
13									
14									
15 3 - access("R"."MBR"="RUK")									
16									
17 Note									
18									
19 - dynamic sampling used for this statement (level=2)									

93

93

Indeksi i strani ključ (ugneždeni sa indeksom i bez indeksa)

```
SELECT PLAN_TABLE_OUTPUT FROM TABLE (DBMS_XPLAN.DISPLAY('plan_table','primer9_ugn'));
```

PLAN_TABLE_OUTPUT									
1 Plan hash value: 166315348									
2									
3									
4									
5									
6	0	SELECT STATEMENT		3	78	3	(0)	00:00:01	
7	1	NESTED LOOPS SEMI		3	78	3	(0)	00:00:01	
8	2	TABLE ACCESS FULL	RADNIK	22	286	3	(0)	00:00:01	
9	3	INDEX RANGE SCAN	IND_FK	1	13	0	(0)	00:00:01	
10									
11									
12 Predicate Information (identified by operation id):									
13									
14									
15 3 - access("R"."MBR"="RUK")									
16									
17 Note									
18									
19 - dynamic sampling used for this statement (level=2)									

```
SELECT PLAN_TABLE_OUTPUT FROM TABLE (DBMS_XPLAN.DISPLAY('plan_table','primer8_ugn'));
```

PLAN_TABLE_OUTPUT									
1 Plan hash value: 4292270174									
2									
3									
4									
5									
6	0	SELECT STATEMENT		3	78	6	(17)	00:00:01	
7	1	MERGE JOIN SEMI		3	78	6	(17)	00:00:01	
8	2	TABLE ACCESS BY INDEX ROWID	RADNIK	22	286	2	(0)	00:00:01	
9	3	INDEX FULL SCAN	RADNIK_PK	22		1	(0)	00:00:01	
10	4	SORT UNIQUE		8	104	4	(25)	00:00:01	
11	5	TABLE ACCESS FULL	PROJEKAT	8	104	3	(0)	00:00:01	
12									
13									
14 Predicate Information (identified by operation id):									
15									
16									
17 4 - access("R"."MBR"="RUK")									
18 filter("R"."MBR"="RUK")									
19									
20 Note									
21									
22 - dynamic sampling used for this statement (level=2)									

94

94

Indeksi i strani ključ (ugneždeni i spoj sa indeksom)

SELECT PLAN_TABLE_OUTPUT FROM TABLE (DBMS_XPLAN.DISPLAY('plan_table','primer9_'))		SELECT PLAN_TABLE_OUTPUT FROM TABLE (DBMS_XPLAN.DISPLAY('plan_table','primer9_spo'))	
Script Output x Autotrace x Explain Plan x Query Result x Query Result 1 x Query Result 2		Script Output x Autotrace x Explain Plan x Query Result x Query Result 1 x Query Result 2	
SQL All Rows Fetched: 19 in 0.02 seconds		SQL All Rows Fetched: 20 in 0.02 seconds	
PLAN_TABLE_OUTPUT 1 Plan hash value: 166315348 2 3 4 Id Operation Name Rows Bytes Cost (%CPU) Time 5 6 0 SELECT STATEMENT 3 78 3 (0) 00:00:01 7 1 NESTED LOOPS SEMI 3 78 3 (0) 00:00:01 8 2 TABLE ACCESS FULL RADNIK 22 286 3 (0) 00:00:01 9 * 3 INDEX RANGE SCAN IND_FK 1 13 0 (0) 00:00:01 10 11 12 Predicate Information (identified by operation id): 13 14 15 3 - access("R"."MBR"="RUK") 16 17 Note 18 ----- 19 - dynamic sampling used for this statement (level=2)		PLAN_TABLE_OUTPUT 1 Plan hash value: 301486184 2 3 4 Id Operation Name Rows Bytes Cost (%CPU) Time 5 6 0 SELECT STATEMENT 8 208 4 (25) 00:00:01 7 1 HASH UNIQUE 8 208 4 (25) 00:00:01 8 2 NESTED LOOPS 8 208 3 (0) 00:00:01 9 3 TABLE ACCESS FULL RADNIK 22 286 3 (0) 00:00:01 10 * 4 INDEX RANGE SCAN IND_FK 1 13 0 (0) 00:00:01 11 12 13 Predicate Information (identified by operation id): 14 15 16 4 - access("R"."MBR"="P"."RUK") 17 18 Note 19 ----- 20 - dynamic sampling used for this statement (level=2)	
95		95	

Primer kompozitnog indeksa

```
CREATE INDEX ind_imeprz ON radnik(ime,prz);
```

```
EXPLAIN PLAN
```

```
SET STATEMENT_ID = 'primer11'
```

```
FOR
```

```
SELECT r.ime, r.prz
```

```
FROM radnik r;
```

```
SELECT PLAN_TABLE_OUTPUT FROM TABLE (DBMS_XPLAN.DISPLAY('plan_table','primer11'));
```

96

Primer kompozitnog indeksa

```
CREATE INDEX ind_cp ON radnik(ime, prz);

EXPLAIN PLAN
SET STATEMENT_ID = 'primer11'
FOR
SELECT r.ime, r.prz
FROM radnik r;

SELECT PLAN_TABLE_OUTPUT FROM TABLE (DBMS_XPLAN.DISPLAY('plan_table', 'primer11'));
```

Script Output x Query... x

SQL | All Rows Fetched: 8 in 0.109 seconds

PLAN_TABLE_OUTPUT

1 Plan hash value: 623867871

2

3 -----

4 | Id | Operation | Name | Rows | Bytes | Cost (%CPU) | Time |

5 -----

6 | 0 | SELECT STATEMENT | | 23 | 276 | 1 (0) | 00:00:01 |

7 | 1 | INDEX FULL SCAN | IND_CP | 23 | 276 | 1 (0) | 00:00:01 |

8 -----

97

97

Dodatno

98

Quinine HM & TonicDB

- Optimizacija na virtualizacionom nivou
 - <https://dataware.dev/>

